
Beyond Permutation Symmetry: Symmetry, Redundancy, and Neural-Network Connectivity

Anonymous Author
Anonymous Institution

Abstract

The single-basin-modulo-permutation conjecture of Entezari et al. (2021) proposes that independently trained neural networks can often be connected by a low-loss linear path after permuting their hidden units. We test the limits of this explanation. Exhaustive enumeration of all hidden-unit permutations in minimal-width XOR networks yields a counterexample among gradient-trained solutions: pairs of successful endpoints can remain linearly incompatible under every permutation. We then consider a larger exact symmetry class. Building on the differentiable Sinkhorn re-basing method of Peña et al. (2023), we augment permutation alignment with positive ReLU scaling. We also introduce a method-agnostic refinement that freezes the permutation returned by any alignment method and subsequently optimizes only the scales. Scale-aware alignment lowers interpolation barriers beyond permutation-only alignment throughout our controlled XOR study and across all evaluated VGG/CIFAR-10 and MLP/Fashion-MNIST architectures. Finally, we study training-stage connectivity by extending the same-stage analysis of Ainsworth et al. (2023) to checkpoints from two independent runs at unequal epochs, $n \neq N$. The worse-endpoint barrier—the maximum interpolation loss in excess of the larger endpoint loss—is negligible early in training and becomes non-negligible only after a distinct training stage. Together, these results show that permutation alone does not exhaust the function-preserving transformations relevant to interpolation and identify when excess in-

terpolation barriers emerge during training.

1 INTRODUCTION

Are independently trained neural networks genuinely in different basins, or do they only appear separated because their hidden features use different coordinates? The question matters both for the geometry of neural-network loss landscapes and for operations such as weight averaging. A high-loss straight line between two parameter vectors is not conclusive: neural-network parameterizations are non-identifiable, and functionally equivalent representatives can have very different interpolation behavior.

The single-basin-modulo-permutation conjecture proposes that most stochastic gradient descent (SGD) solutions can be permuted into linear mode connectivity (LMC) (Entezari et al., 2021). Re-basing methods make this hypothesis operational and substantially reduce barriers in MLPs and convolutional networks (Ainsworth et al., 2023; Peña et al., 2023). Yet a residual barrier after alignment is ambiguous. An algorithm may have missed a suitable permutation, the permutation symmetry class may itself be too small, or the architecture may lack the representational freedom needed for a low-loss path. Large-network studies cannot cleanly distinguish these explanations because their permutation groups are combinatorial and alignment is approximate.

We isolate these mechanisms using XOR networks small enough for exhaustive permutation search, then test the resulting account in VGG networks. The minimal-width experiment first removes the search confound: some trained pairs remain classification-incompatible under every permutation. We reproduce this behavior with full-batch gradient descent (GD) and minibatch SGD. We then enlarge the exact symmetry class using positive ReLU scaling. Starting from the exhaustive best permutation and optimizing scales alone identifies gains that cannot be attributed to better correspondence search. Finally, nonlinear path and width

experiments separate representative choice from architectural redundancy: symmetry moves within an equivalence class, whereas extra units create additional routes through parameter space.

Our contributions are:

- We give an exhaustive counterexample to universal permutation-based LMC for trained networks. At minimal XOR width, 22/28 successful pairs fail classification-LMC after the best permutation under both full-batch GD and minibatch SGD.
- We introduce scale-aware alignment and show, under an exhaustive permutation oracle, that positive scaling improves or preserves the loss barrier for all 67 tested XOR pairs at widths 3, 5, and 7.
- We show an empirical transition in nonlinear connectivity between widths two and three, distinguishing representational redundancy from symmetry alignment.
- We show that alignment difficulty depends on shared training history and that scaling improves over permutation-only alignment on fixed VGG11/13/16/19 pairs trained on CIFAR-10.

2 PROBLEM FORMULATION AND RELATED WORK

Connectivity and interpolation barriers. For endpoints θ_A and θ_B , linear interpolation is $\theta(t) = (1 - t)\theta_A + t\theta_B$. Following prior work, for a scalar criterion C (loss or classification error) we measure the excess above the linear interpolation of endpoint values:

$$\mathcal{B}(\theta_A, \theta_B; C) = \max_{t \in [0, 1]} [C(\theta(t)) - (1 - t)C(\theta_A) - tC(\theta_B)]. \quad (1)$$

The maximum is evaluated on a finite grid in our experiments. Nonlinear mode connectivity instead asks whether a general curve $\gamma : [0, 1] \rightarrow \Theta$ joins the endpoints at low loss. A curved path may avoid a region crossed by the straight segment, so nonlinear connectivity does not imply LMC (Draxler et al., 2018; Garipov et al., 2018).

Permutation re-basing. For a ReLU MLP block $f(x) = W_2\sigma(W_1x)$ and a permutation matrix P ,

$$(W_2P^\top)\sigma(PW_1x) = W_2\sigma(W_1x). \quad (2)$$

The same action reorders channels in convolutional networks. Weight matching finds layerwise permutations that reduce Euclidean distance, activation matching

aligns representations on data, and differentiable methods optimize soft assignments (Ainsworth et al., 2023; Peña et al., 2023). REPAIR shows that correcting activation statistics can further improve interpolation after permutation (Jordan et al., 2023). These successes support a symmetry-aware view of loss landscapes, but a residual barrier obtained by a heuristic cannot establish that the symmetry class itself is insufficient.

Why linear connectivity matters. A curved connection establishes geometric connectedness, but a straight path is the relevant object for parameter averaging. Checkpoint averaging can improve generalization within one trajectory (Izmailov et al., 2019), and averaging related fine-tuned models can be effective without ensemble-time cost (Wortsman et al., 2022). Our focus is the geometric prerequisite—a low-barrier segment—rather than a claim that an aligned midpoint must outperform its endpoints.

3 SCALE-AWARE ALIGNMENT

Permutation is only one part of the exact symmetry group of a ReLU network. Positive homogeneity, $\sigma(cz) = c\sigma(z)$ for $c > 0$, permits the incoming weights and bias of a unit to be multiplied by c when its outgoing weight is divided by c . In matrix form, for positive diagonal D ,

$$(W_2D^{-1})\sigma(DW_1x) = W_2\sigma(W_1x). \quad (3)$$

Permutation and scaling therefore form an exact monomial action. Scaling does not change either endpoint function, but it changes the cross-model terms created inside a linear parameter interpolation and can therefore change the barrier. This motivates studying connectivity under a richer symmetry class before attributing a residual barrier to genuinely distinct basins.

For scalable optimization, we relax each hard permutation to

$$S_\ell = \text{Sinkhorn}(Z_\ell/\tau), \quad D_\ell = \text{diag}(e^{u_\ell}), \quad (4)$$

where Z_ℓ contains unconstrained assignment scores and u_ℓ contains log-scales. On layer- ℓ outputs we apply $S_\ell D_\ell$ and on the next layer’s inputs the compensating $D_\ell^{-1} S_\ell^\top$. After projecting S_ℓ to a hard assignment, this is exactly function preserving. During soft optimization, the transpose is the standard differentiable Sinkhorn approximation rather than an exact inverse.

We consider two variants. *Joint permutation+scale* optimizes Z_ℓ and u_ℓ together. *Scale refinement* first obtains and freezes a hard permutation, then optimizes u_ℓ alone. On XOR, the frozen permutation can be the

exhaustive optimum, making the incremental contribution of scale identifiable. A quadratic penalty on u_ℓ controls the unbounded scale gauge.

The optimization objective is experiment-specific. For XOR we directly minimize a differentiable estimate of Eq. 1: the maximum excess binary cross-entropy over 31 interpolation points, plus log-scale regularization. For VGG, evaluating the full path at every update is costly, so alignment minimizes calibration loss at the midpoint; reported barriers are then computed from the test-set interpolation after hard projection. This distinction avoids conflating the scalable proxy used on VGG with the barrier objective used in the controlled experiment.

4 EXPERIMENT I: PERMUTATION IS NOT ENOUGH

Question. When permutation-based LMC fails, did the alignment algorithm fail, or does no permutation work? We answer this in the smallest nontrivial setting: a one-hidden-layer ReLU network of width $h = 2$ trained on the four XOR examples. There are only $2! = 2$ hidden-unit permutations, so the symmetry class can be searched exactly.

Training and evaluation. Models are initialized independently and retained only if they reach 100% endpoint accuracy and low binary cross-entropy. Our primary protocol uses full-batch GD. As a robustness check against the interpretation that the effect is specific to deterministic optimization, we repeat the experiment with minibatch SGD using batch size two and reshuffling each epoch. For each protocol, eight successful endpoints give 28 pairs. We enumerate identity and swap and evaluate classification error along each interpolation.

Result. For both training protocols, only 6/28 pairs preserve 100% accuracy along the best-permuted segment; the remaining 22/28 incur interpolation error under both possible permutations. Figure 1 shows a successful pair and a representative full-batch failure. In the failure, both endpoints correctly implement XOR and have similar decision boundaries, but no hidden-unit relabeling makes their straight interpolation compatible. Because the search is exhaustive, this result cannot be attributed to weight matching, Sinkhorn relaxation, or alignment initialization.

The conclusion is deliberately scoped. These trained endpoints refute a universal claim that permutation-based LMC follows automatically from SGD or GD. They do not contradict the sufficiently-wide, high-probability form of the single-basin-modulo-

permutation conjecture. They also complement the hand-constructed counterexample of Ainsworth et al. (2023): LMC failure is not confined to endpoints chosen algebraically outside a standard training procedure.

5 EXPERIMENT II: A RICHER SYMMETRY EXPLAINS MORE LMC

Question and setup. Does positive scaling improve compatibility once permutation-search error has been removed? We train full-batch GD XOR networks at widths $h = 3, 5, 7$, giving 21, 36, and 10 valid endpoint pairs. For each pair we enumerate all $h!$ permutations and select the one with the smallest sampled loss barrier. We then freeze this oracle permutation and optimize only positive scales. Final barriers use 61 interpolation points. We also report raw interpolation and practical Sinkhorn baselines. Additional optimization details appear in Appendix A.

5.1 Width lowers the oracle barrier

Increasing width changes the result. In the initial exhaustive sweep over $h \in \{2, 3, 4, 5\}$, the number of pairs maintaining 100% accuracy along the best-permuted interpolation rises steadily; at $h = 5$, every evaluated pair maintains perfect accuracy. Perfect accuracy does not mean zero loss barrier: midpoint confidence can still fall relative to the endpoints.

The larger scale-aware study quantifies this effect in loss. Table 1 shows that the oracle permutation barrier falls from 0.357 at $h = 3$ to 0.111 at $h = 5$ and 0.023 at $h = 7$. Thus, even before scaling, width supplies matching flexibility. Practical Sinkhorn permutation search is weaker than the oracle at all widths, illustrating why heuristic failure alone cannot diagnose the absence of an alignment.

Scale beyond the oracle permutation. The decisive comparison in Table 1 is between oracle permutation and oracle+scale. The hard assignment is identical in the two conditions, so the improvements can only come from selecting a different functionally equivalent scaling of the same endpoint. Refinement reduces the mean barrier by 31% at $h = 3$, 77% at $h = 5$, and 78% at $h = 7$. It is no worse on every one of the 67 pairs. This shows that positive scale is not merely helping a relaxed optimizer discover a better permutation; it enlarges the attainable set beyond the exhaustive permutation orbit.

Joint optimization is more mixed at the narrowest width. It improves over the permutation oracle on 9/21, 29/36, and 8/10 pairs at widths 3, 5, and 7, respectively.

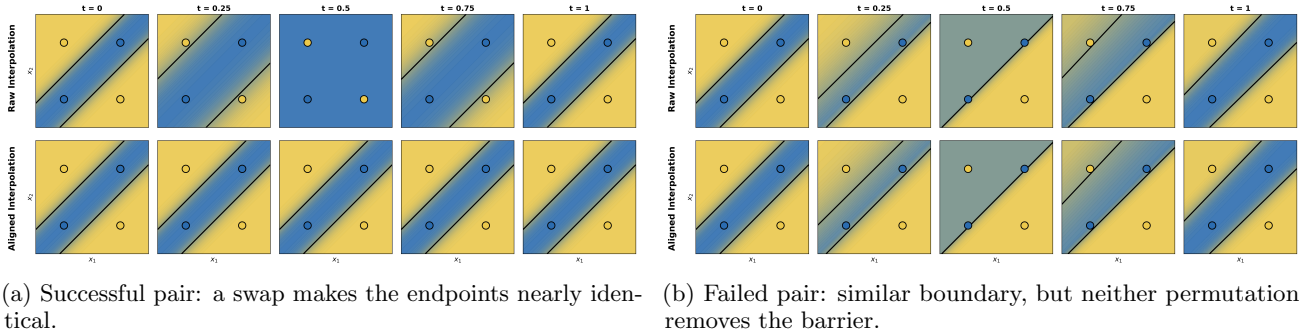


Figure 1: Exhaustive permutation alignment for width-2 XOR networks. Columns in each panel are interpolation points; top and bottom rows show raw and best-permuted interpolation. Both endpoints classify all four XOR points correctly.

Table 1: XOR loss barrier (mean $\pm$ standard deviation). Oracle+scale first selects the exhaustive best permutation, then optimizes only positive scales. The final row counts pairs for which refinement is no worse than the oracle.

Method	$h = 3$	$h = 5$	$h = 7$
No alignment	0.780 ± 0.610	0.608 ± 0.465	0.331 ± 0.184
Sinkhorn permutation	0.444 ± 0.325	0.189 ± 0.162	0.063 ± 0.044
Oracle permutation	0.357 ± 0.309	0.111 ± 0.118	0.023 ± 0.015
Joint permutation+scale	0.428 ± 0.380	0.074 ± 0.149	0.009 ± 0.009
Oracle+scale	0.247 ± 0.283	0.025 ± 0.044	0.005 ± 0.003
Oracle+scale $\leq$ oracle	21/21	36/36	10/10

At $h = 3$, its mean barrier (0.428) remains above the oracle-permutation result (0.357), whereas at widths five and seven it beats the oracle mean. The difference between joint and oracle-refinement results exposes an optimization gap: a richer symmetry class can contain better representatives without a from-scratch solver reliably finding them.

Figure 2 shows the averaged interpolation profiles. All curves improve with width, but oracle scale refinement (green) is consistently lowest. At $h = 7$, its mean maximum excess is only 0.005, compared with 0.023 for the best permutation. Scaling matters most once width has already made useful correspondences available: it fine-tunes the balance of incoming and outgoing weights without changing endpoint predictions.

6 EXPERIMENT III: WIDTH CHANGES NONLINEAR CONNECTIVITY

Question and setup. Residual linear barriers need not indicate disconnected low-loss sets. To test whether width affects available routes rather than only alignment, we optimize quadratic Bézier curves and polygonal chains between independently trained XOR endpoints. Paths use up to ten trainable interior points and minimize the maximum loss over 31 sampled path

locations.

Result. At $h = 2$, extensive searches find low-loss nonlinear paths only between essentially identical endpoints; additional bends do not recover nontrivial connections. At $h = 3$, all studied endpoint pairs are connected by low-loss paths. Most are found directly; for one difficult pair, concatenating low-loss paths through a third trained mode establishes connectivity. Figure 3 contrasts representative outcomes.

This result is empirical, not a proof that no width-2 path exists outside the tested families. Nevertheless, the abrupt change after adding one unit supports a distinction that endpoint alignment alone cannot make: symmetry selects among equivalent representatives at fixed capacity, while width changes the representational resources available along a path.

7 EXPERIMENT IV: AN EARLY TRANSITION IN TRAINING CONNECTIVITY

Setup. Two VGG16/CIFAR-10 branches share an initialization and the first n of $N = 200$ epochs, then train independently with different minibatch orders and augmentations. A known random channel permutation is applied to one endpoint, and weight matching

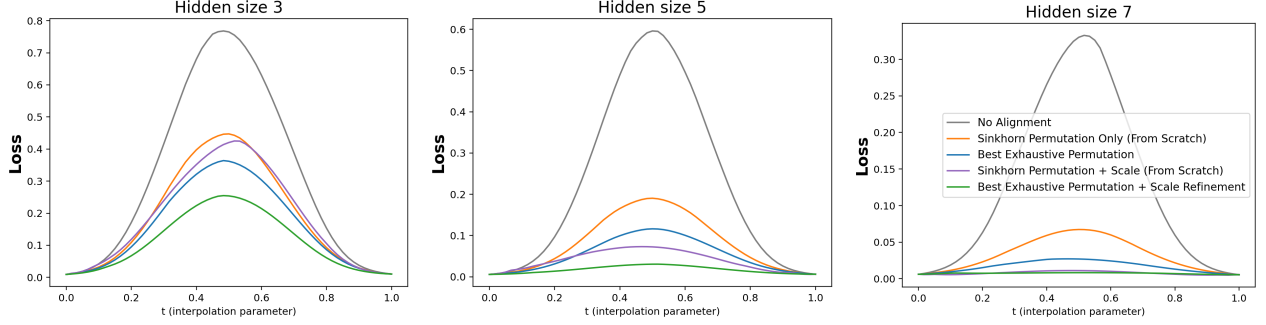


Figure 2: Mean XOR interpolation loss at widths 3, 5, and 7: raw interpolation (gray), Sinkhorn permutation (orange), exhaustive oracle permutation (blue), joint permutation+scale (purple), and oracle permutation followed by scale refinement (green). Exact barriers are in Table 1.

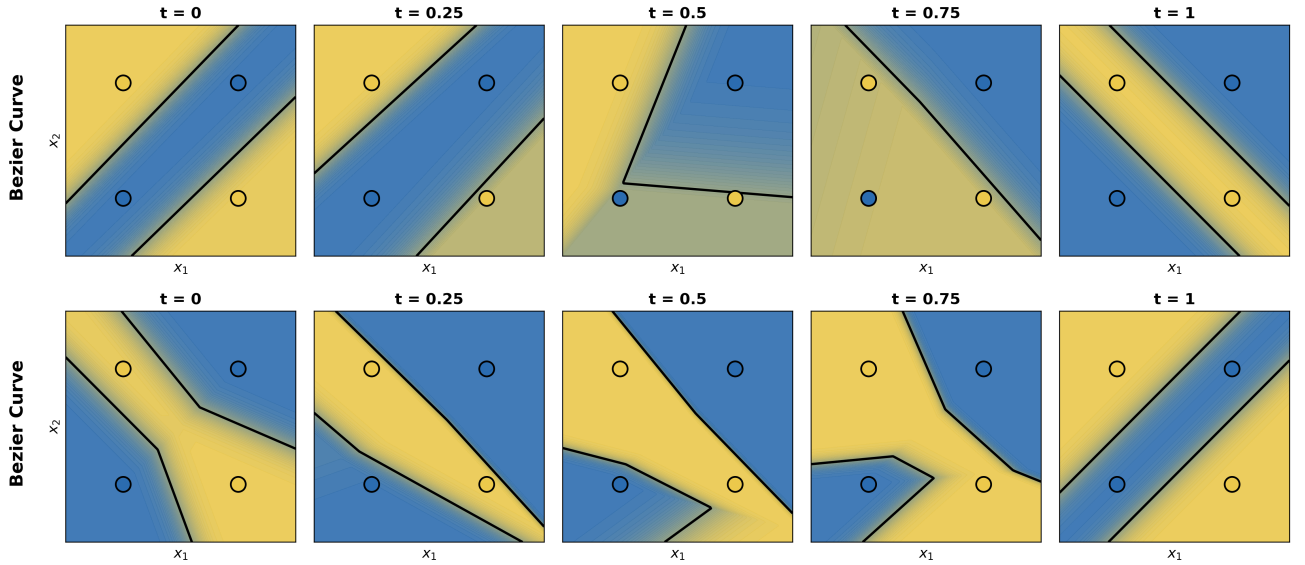


Figure 3: Nonlinear XOR paths. *Top*: a representative width-2 pair for which optimizing a Bézier path does not remove the high-loss region. *Bottom*: at width 3, a low-loss path connects endpoints with different decision boundaries. Rows visualize predictions along the learned path.

attempts to recover compatibility. The inverse planted permutation supplies a reference, while repeated layer-update orders measure search variability. This design varies shared training history; it is not a checkpoint-by-checkpoint decomposition of raw, scale, and nonlinear barriers.

Result. Figure 4 plots test-error barrier against end-point distance. Late splits lie in the lower-left: their endpoints remain close, the reference path has low barrier, and matching reliably removes the planted mismatch. Early splits lie farther away and retain larger barriers.

A denser sweep over the first training epochs localizes a sharp transition in this run. After alignment, the test-loss barrier is 0.120 when the branches share five epochs and train independently for 195, but falls to

0.001 for the 6/194 split and remains near zero for all later splits. Thus, sharing one additional early epoch qualitatively changes the linear geometry of the final solutions. Branches separated after this transition remain in a common linear basin modulo symmetry despite different subsequent minibatches and augmentations; branches separated earlier develop residual barriers that permutation alignment does not remove.

For every shared-split pair, the recovered alignment is at least as good as the known inverse permutation and often better. Repeating weight matching with different layer-update orders produces small variability (the largest barrier standard deviation is about 0.68 percentage points for the 0/200 split and becomes negligible for late splits). The trend is therefore not explained by one unlucky search order.

Distance is informative but not sufficient. The 4/196 shared-split pair has a test-error barrier of approximately 21.7 percentage points near distance 28, whereas the independently initialized pair after matching has a barrier of 34.1 points at a comparable distance. The experiment suggests that weight similarity is a useful proxy when trajectories share substantial history, but ceases to characterize linear compatibility once their learned representations diverge. This motivates optimizing interpolation loss within a broader symmetry orbit, as in Section 3. The result identifies a transition in linear compatibility; because curved paths were not optimized for every shared-split pair, it does not by itself establish that the residual pairs are connected only nonlinearly.

8 EXPERIMENT V: VGG/CIFAR-10 VALIDATION

Setup. We use fixed, independently trained seed-0/seed-1 pairs of VGG11, VGG13, VGG16, and VGG19. Alignment uses a deterministic 1000-example calibration batch. We compare raw interpolation, permutation-only Sinkhorn, joint permutation+scale, and hard permutation followed by scale refinement; test-set barriers are evaluated after hard projection.

Result. Figure 5 reports test-loss barriers for the four architectures. Permutation-only Sinkhorn reduces the raw barriers from roughly 1.88–1.98 to 0.38, 0.79, 1.22, and 1.21 for VGG11/13/16/19. Adding scales improves on permutation-only in every architecture. The better of joint and sequential scaling attains 0.109, 0.429, 1.035, and 1.105, respectively. Relative to permutation-only, these are reductions of about 71%, 45%, 15%, and 9%.

Neither optimization order dominates. Joint permutation+scale is better for VGG11 and VGG19; freezing the permutation and refining scales is better for VGG13 and VGG16. This echoes XOR: the symmetry class contains better representatives, but joint nonconvex optimization does not always realize the full gain. Substantial barriers also remain for deeper VGGS, so permutation and scale together are not a complete account of LMC.

The VGG results should be read as a transfer check of the controlled XOR finding. Exhaustive permutation search is impossible, only one endpoint pair is used per architecture, and hyperparameters are selected per method. Nevertheless, the consistent direction of change across four architectures supports the practical relevance of scaling beyond the toy setting.

9 DISCUSSION AND LIMITATIONS

Symmetry and redundancy play different roles.

The experiments support a two-part account of apparent basin separation. Permutation removes discrete coordinate mismatch, and positive scaling moves continuously along the same exact symmetry orbit. Neither changes the endpoint function. Width, by contrast, changes the architecture’s representational freedom and can create low-loss routes unavailable to a narrower network. Curved paths further relax the requirement that this route be the straight segment. These mechanisms interact, but they are not interchangeable: a better representative cannot manufacture absent capacity, and additional capacity does not identify the right representative by itself.

The minimal-width experiment also sharpens the interpretation of Ainsworth et al. (2023). Their hand-constructed counterexample shows that architecture alone does not guarantee permutation-based LMC, and they suggest that LMC may be an artifact of SGD. Our result rules out a broad reading under which standard gradient-based training itself guarantees alignable endpoints: neither minibatch SGD nor full-batch GD prevents permutation-incompatible successful solutions. It does not rule out an SGD bias toward compatibility in wider or typical modern networks. Indeed, the rapid fall in the oracle barrier with width is consistent with the sufficiently-wide, high-probability form of the single-basin-modulo-permutation conjecture (Entezari et al., 2021; Ainsworth et al., 2023).

Optimization versus existence. The exhaustive protocol is central to this interpretation. Without it, a high barrier after Sinkhorn or weight matching could always be ascribed to search failure. Conversely, the gap between joint Sinkhorn and oracle refinement warns against treating a failed continuous relaxation as evidence that a good monomial representative does not exist. Future work should distinguish the expressivity of a transformation class from the solver used to search it.

Implications for alignment studies. A residual barrier should be interpreted in stages. Search error must first be separated from limitations of the transformation class. If permutation is insufficient, richer exact symmetries should be tested before inferring distinct basins. Linear and nonlinear paths should then be distinguished, and width should be treated as a change in representational resources rather than merely a larger matching problem. The training-history experiment adds a final caution: comparable Euclidean distances can conceal different barriers, so distance is not a certificate of functional compatibility.

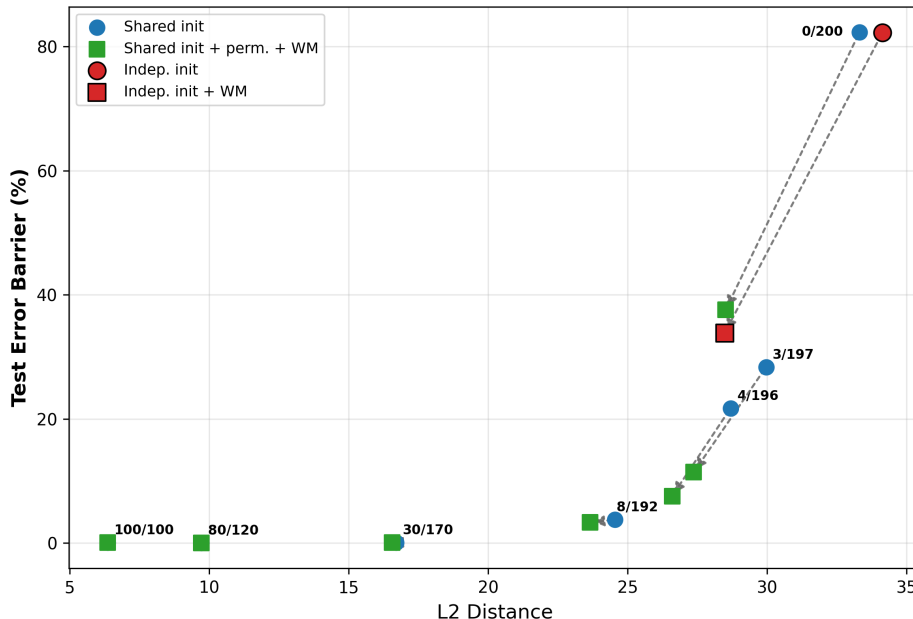


Figure 4: VGG16/CIFAR-10 shared-training benchmark. Labels $n/(200 - n)$ denote shared/independent epochs. Circles are original endpoints; squares are endpoints after applying a random permutation and weight matching. Red denotes independent initialization. Later splits remain closer and have lower barriers after alignment; the dense early-epoch sweep discussed in the text places the sharp loss-barrier transition between the 5/195 and 6/194 splits.

Limitations. The exact conclusions come from small one-hidden-layer XOR networks. The minibatch and full-batch protocols are robustness replications, not a controlled estimate of stochasticity: with the same epoch count, batch size two performs more parameter updates than full-batch GD. The VGG study is non-exhaustive, uses a single pair per architecture, and relies on method-specific hyperparameter selection. Our non-linear path searches can certify found paths but cannot certify nonexistence. The scale-aware soft transformation is exactly function preserving only after projection; during Sinkhorn optimization it is a relaxation. Finally, we consider ReLU networks, for which positive homogeneity holds, and do not address normalization layers, residual architectures, attention symmetries, or approximate functional equivalences. Larger multi-seed studies and comparisons with activation repair are needed before drawing broad architectural conclusions.

10 CONCLUSION

Permutation symmetry is an important but incomplete explanation of neural-network connectivity. Exhaustive XOR search shows that trained endpoints can remain incompatible under every permutation, for both minibatch SGD and full-batch GD. Positive scaling improves even the exhaustive best permutation, while the width-dependent nonlinear experiment

shows that exact symmetries do not subsume representational constraints. The same scaling effect transfers to fixed VGG/CIFAR-10 pairs, and shared-training experiments show that compatibility also depends on optimization history. The central distinction is therefore simple: symmetry determines which representatives we compare; redundancy determines which paths are available.

AI Use Statement

Generative AI tools were used to brainstorm presentation choices, improve the organization and readability of draft text, and assist with programming tasks including debugging, refactoring, plotting, and $\text{\LaTeX}$ editing. They were not used to generate experimental data, select the reported results, or make the final scientific interpretations. The research questions, experimental design, implementation decisions, analysis, and conclusions were determined by the authors. All AI-assisted text and code were reviewed and checked before inclusion. The authors take responsibility for the final content, claims, citations, and artifacts. This statement must be reconciled with the complete author team’s actual use before submission.

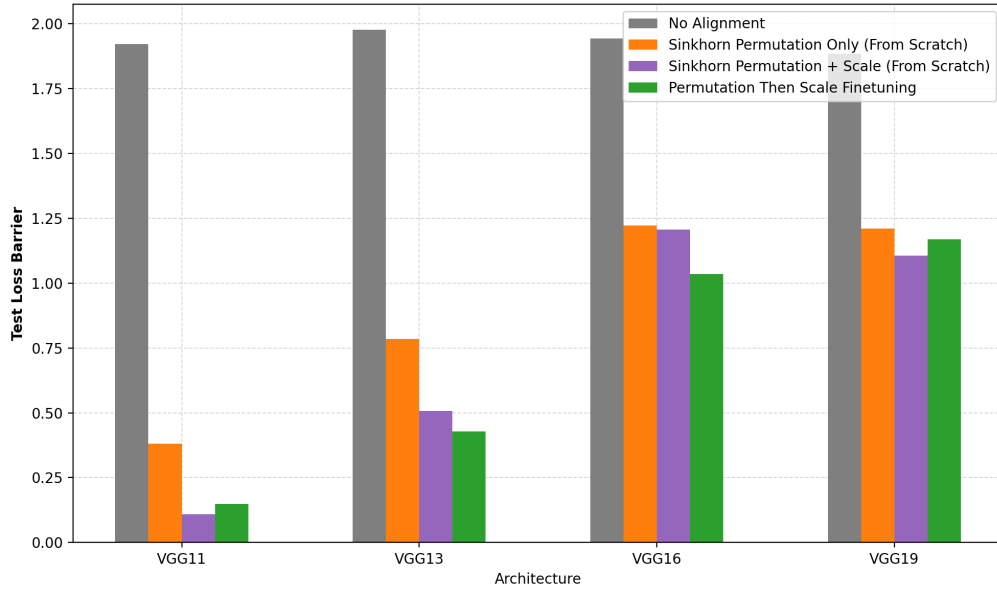


Figure 5: Maximum test-loss barrier for independently trained VGG models on CIFAR-10. Both scale-aware variants improve on Sinkhorn permutation-only for all four fixed endpoint pairs, although joint and sequential optimization trade places across architectures.

References

- Ainsworth, S., Hayase, J., and Srinivasa, S. (2023). Git re-basin: Merging models modulo permutation symmetries. In *The Eleventh International Conference on Learning Representations*.
- Draxler, F., Veschgini, K., Salmhofer, M., and Hamprecht, F. (2018). Essentially no barriers in neural network energy landscape. In Dy, J. and Krause, A., editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1309–1318. PMLR.
- Entezari, R., Sedghi, H., Saukh, O., and Neyshabur, B. (2021). The role of permutation invariance in linear mode connectivity of neural networks. In *International Conference on Learning Representations*.
- Garipov, T., Izmailov, P., Podoprikin, D., Vetrov, D. P., and Wilson, A. G. (2018). Loss surfaces, mode connectivity, and fast ensembling of DNNs. In *Advances in Neural Information Processing Systems*, volume 31.
- Izmailov, P., Podoprikin, D., Garipov, T., Vetrov, D., and Wilson, A. G. (2019). Averaging weights leads to wider optima and better generalization.
- Jordan, K., Sedghi, H., Saukh, O., Entezari, R., and Neyshabur, B. (2023). REPAIR: REnormalizing permuted activations for interpolation repair. In *International Conference on Learning Representations*.
- Peña, F. A. G., Medeiros, H. R., Dubail, T., Aminbeidokhti, M., Granger, E., and Pedersoli, M. (2023). Re-basin via implicit sinkhorn differentiation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20237–20246.
- Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A. S., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S., and Schmidt, L. (2022). Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S., editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 23965–23998. PMLR.

CHECKLIST

For each question, the selected answer is shown in brackets, followed where useful by a short justification.

- For all models and algorithms presented, check if you include:
 - A clear description of the mathematical setting, assumptions, algorithm, and/or model. [Yes; Sections 2–3 and Appendix A.]
 - An analysis of the properties and complexity (time, space, sample size) of any algorithm.

- [No; empirical behavior rather than asymptotic complexity is the focus.]
 - (c) (Optional) Anonymized source code, with specification of all dependencies, including external libraries. [No.]
2. For any theoretical claim, check if you include:
- (a) Statements of the full set of assumptions of all theoretical results. [Not Applicable; no formal theorem is claimed.]
 - (b) Complete proofs of all theoretical results. [Not Applicable.]
 - (c) Clear explanations of any assumptions. [Yes; the positive-homogeneity assumption is stated in Section 3.]
3. For all figures and tables that present empirical results, check if you include:
- (a) The code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL). [No; experimental settings are included, but an anonymized code release is not yet linked.]
 - (b) All the training details (e.g., data splits, hyperparameters, how they were chosen). [Yes; Appendix A.]
 - (c) A clear definition of the specific measure or statistics and error bars (e.g., with respect to the random seed after running experiments multiple times). [Yes; Eq. 1, Table 1, and the limitations in Section 8.]
 - (d) A description of the computing infrastructure used. (e.g., type of GPUs, internal cluster, or cloud provider). [No.]
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets, check if you include:
- (a) Citations of the creator If your work uses existing assets. [Yes.]
 - (b) The license information of the assets, if applicable. [No.]
 - (c) New assets either in the supplemental material or as a URL, if applicable. [Not Applicable.]
 - (d) Information about consent from data providers/curators. [Not Applicable; only synthetic XOR and the public CIFAR-10 benchmark are used.]
 - (e) Discussion of sensible content if applicable, e.g., personally identifiable information or offensive content. [Not Applicable.]
5. If you used crowdsourcing or conducted research with human subjects, check if you include:
- (a) The full text of instructions given to participants and screenshots. [Not Applicable.]
 - (b) Descriptions of potential participant risks, with links to Institutional Review Board (IRB) approvals if applicable. [Not Applicable.]
 - (c) The estimated hourly wage paid to participants and the total amount spent on participant compensation. [Not Applicable.]

Beyond Permutation Symmetry: Symmetry, Redundancy, and Neural-Network Connectivity — Supplementary Materials

A REPRODUCIBILITY DETAILS

A.1 XOR training and exhaustive alignment

The XOR set is $\{(0, 0), (0, 1), (1, 0), (1, 1)\}$ with labels $\{0, 1, 1, 0\}$. Models have one ReLU hidden layer, biases, and one output logit. The primary endpoint protocol uses full-batch GD (implemented with the SGD optimizer) at learning rate 0.05 for at most 5000 epochs. `ReduceLROnPlateau` uses factor 0.5, patience 200, threshold 10^{-4} , and minimum learning rate 5×10^{-4} . Training stops when loss has not improved by 10^{-4} for 500 epochs. For each retained pair, all $h!$ permutations are evaluated. Interpolation selection prioritizes classification error in the width-2 diagnostic and loss barrier in the scale-aware study.

The width-2 robustness replication uses true minibatch SGD with batch size two and reshuffles the four XOR examples every epoch. It otherwise uses the same initialization range, learning-rate schedule, stopping rule, maximum of 5000 epochs, accepted seeds, and exhaustive identity/swap evaluation. Eight successful endpoints produce 28 pairs in each protocol. Matching epoch counts does not match optimizer-update counts: the minibatch protocol makes two updates per epoch and is therefore a robustness comparison, not an isolated causal test of stochastic gradient noise. The width-3/5/7 scale-aware study uses the primary full-batch-GD endpoints.

For nonlinear paths, both quadratic Bézier curves and polygonal chains are optimized for 1500 steps at learning rate 0.05, with no weight decay and 31 path samples. The default path has one trainable interior point; harder pairs use up to ten.

A.2 XOR scale-aware optimization

Final interpolation curves use 61 evaluation points. XOR alignment directly optimizes the maximum sampled excess binary cross-entropy over 31 interpolation points, rather than midpoint loss alone. The default permutation-only Sinkhorn run uses Adam for 500 steps, learning rate 0.01, temperature 1, 20 Sinkhorn iterations, and identity strength 1. Scale refinement uses Adam for 500 steps, learning rate 0.01, and log-scale regularization 10^{-3} . Joint optimization uses the same default budget. If a default run misses its target, a bounded search over learning rates, temperatures, and regularization is used; settings are selected per pair. The scale target is barrier $\epsilon = 0.01$, while permutation-only Sinkhorn targets that pair’s exhaustive barrier.

A.3 VGG experiments

The shared-training benchmark follows a 200-epoch VGG16/CIFAR-10 schedule. Branches differ in minibatch order and stochastic augmentation after the split. The evaluated shared-training durations are $n \in \{0, 1, 2, 3, 4, 5, 6, 8, 30, 80, 100\}$ epochs; the figure displays a readable subset, while the dense early sweep is used to localize the transition reported in Section 7. The artificial full-network permutation uses seed 42; weight matching uses at most 100 block-coordinate iterations and is repeated with three layer-update orders.

For the scale-aware comparison, fixed seed-0/seed-1 endpoint pairs are used for VGG11/13/16/19. Alignment uses a deterministic batch of 1000 images, 20 Sinkhorn iterations, and midpoint loss. Separate bounded sweeps select the number of optimization steps, learning rate, temperature, and scale regularization for each architecture and method. The permutation-then-scale baseline freezes the projected hard permutation and optimizes only log-scales.

B ADDITIONAL PAIRWISE RESULTS

Table 2 separates the practical optimization gap from the gain provided by the symmetry class. Sinkhorn permutation-only exactly matches the exhaustive oracle on progressively fewer pairs as width grows. Joint permutation+scale often beats the permutation oracle, but oracle scale refinement is the most reliable condition.

Table 2: Pairwise XOR comparisons. Lower loss barrier is better.

Comparison	$h = 3$	$h = 5$	$h = 7$
Oracle+scale $\leq$ oracle permutation	21/21	36/36	10/10
Joint permutation+scale $\leq$ oracle permutation	9/21	29/36	8/10
Sinkhorn permutation = oracle permutation	9/21	7/36	0/10
Oracle+scale $\leq$ joint permutation+scale	15/21	25/36	5/10